

Volcano / PyTorch / NCCL / RDMA

集群中启动 PyTorch 分布式训练说明

目录

1. 前置条件
2. 镜像准备
3. 计算网 IB 设备选择
4. VCJob 示例
5. 启动任务
6. 查看训练日志
7. 确认任务完成
8. 常见调整项
9. 清理任务

本文档说明如何在当前 Kubernetes 集群中使用 Volcano VCJob 启动 PyTorch 分布式训练任务。示例以 2 个节点、每个节点 8 张 GPU、每个 Pod 申请 `rdma/hca: 1` 为基准。

1. 前置条件

集群需要具备以下能力：

- Kubernetes 集群可用。
- Volcano 已安装，并支持 `batch.volcano.sh/v1alpha1` 的 `Job` 资源。
- 集群中存在可用 Volcano queue，例如 `default`。
- GPU 节点已安装 NVIDIA 驱动、NVIDIA device plugin。
- GPU 节点已安装 RDMA device plugin，并暴露 `rdma/hca` 资源。
- 训练镜像已经可以被节点拉取，或者已经提前导入节点 containerd。
- PyTorch 镜像内包含 CUDA、NCCL、PyTorch 分布式训练运行环境。

检查 Volcano 资源：

```
kubectl api-resources --api-group=batch.volcano.sh
kubectl get queue
```

检查节点 GPU/RDMA 资源：

```
kubectl get nodes -o custom-  
columns=NAME:.metadata.name,GPU:.status.allocatable.nvidia\\.com/gpu,RDMA:.status.alloca  
table.rdma/hca
```

2. 镜像准备

示例使用镜像：

```
nvcr.io/nvidia/pytorch:24.01-py3
```

如果节点可以直接访问镜像仓库，只需要在 VCJob 中使用该镜像即可。

如果节点无法直接拉取镜像，可以先在本地拉取，再传到目标节点并导入 containerd：

```
docker pull --platform linux/amd64 nvcr.io/nvidia/pytorch:24.01-py3  
docker save nvcr.io/nvidia/pytorch:24.01-py3 | gzip -1 > /private/tmp/nvidia-pytorch-  
24.01-py3-amd64.tar.gz  
  
scp /private/tmp/nvidia-pytorch-24.01-py3-amd64.tar.gz 10.184.0.44:/tmp/  
scp /private/tmp/nvidia-pytorch-24.01-py3-amd64.tar.gz 10.184.0.46:/tmp/  
  
ssh 10.184.0.44 'gzip -dc /tmp/nvidia-pytorch-24.01-py3-amd64.tar.gz | sudo ctr -n  
k8s.io images import -'  
ssh 10.184.0.46 'gzip -dc /tmp/nvidia-pytorch-24.01-py3-amd64.tar.gz | sudo ctr -n  
k8s.io images import -'
```

确认镜像已在节点 containerd 中：

```
ssh 10.184.0.44 'sudo ctr -n k8s.io images ls | grep nvcr.io/nvidia/pytorch'  
ssh 10.184.0.46 'sudo ctr -n k8s.io images ls | grep nvcr.io/nvidia/pytorch'
```

3. 计算网 IB 设备选择

如果节点同时存在计算网 IB 和存储网 IB，不建议让 NCCL 自由枚举所有 `mlx5_*` 设备。应先确认计算网对应哪些 HCA：

```
ibdev2netdev
```

或者：

```
for d in /sys/class/infiniband/mlx5_*; do
    echo "== $(basename $d) =="
    ls $d/device/net 2>/dev/null
    cat $d/ports/1/link_layer
    cat $d/ports/1/state
done
```

确认计算网 HCA 后，在 VCJob 中通过 `NCCL_IB_HCA` 使用白名单，例如：

```
env:
- name: NCCL_IB_HCA
  value: mlx5_0,mlx5_1,mlx5_2,mlx5_3
```

如果暂时没有区分计算网和存储网，也可以使用：

```
env:
- name: NCCL_IB_HCA
  value: mlx5
```

但生产任务建议使用计算网 HCA 白名单，避免误用存储网设备。

4. VCJob 示例

以下示例会启动一个 2 节点 x 8 卡的 PyTorch 分布式训练任务：

- namespace: `vcjob-rdma-demo`
- job name: `pytorch-2x8-rdma-hca1-demo`
- master: 1 个 Pod
- worker: 1 个 Pod
- 每个 Pod 使用 8 张 GPU
- 每个 Pod 申请 `rdma/hca: 1`
- 每个 Pod 内通过 `torchrun --nproc_per_node=8` 启动 8 个训练进程
- 总 world size: `2 x 8 = 16`

使用前请根据实际集群修改 `nodeAffinity` 节点列表和 `NCCL_IB_HCA` 。

```
apiVersion: v1
kind: Namespace
metadata:
  name: vcjob-rdma-demo
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: pytorch-2x8-rdma-hca1-demo
  namespace: vcjob-rdma-demo
data:
  train.py: |
    import os
    import socket
    import subprocess
    import sys

    import torch
    import torch.distributed as dist

    def run(cmd):
        print(f"$ {cmd}", flush=True)
        try:
            out = subprocess.check_output(cmd, shell=True, stderr=subprocess.STDOUT,
text=True)
            print(out, flush=True)
        except subprocess.CalledProcessError as exc:
            print(exc.output, flush=True)

    rank = int(os.environ["RANK"])
    local_rank = int(os.environ["LOCAL_RANK"])
    world_size = int(os.environ["WORLD_SIZE"])

    torch.cuda.set_device(local_rank)
    if local_rank == 0:
        print("=== PyTorch 2x8 RDMA demo ===", flush=True)
        print(f"host={socket.gethostname()} rank={rank} world_size={world_size}",
flush=True)
        print(f"master={os.environ.get('MASTER_ADDR')}:{os.environ.get('MASTER_PORT')}",
flush=True)
```

```

    print(f"torch={torch.__version__} cuda_available={torch.cuda.is_available()}",
flush=True)

    print(f"cuda_device_count={torch.cuda.device_count()}", flush=True)
    print(f"NCCL_DEBUG={os.environ.get('NCCL_DEBUG')}", flush=True)
    print(f"NCCL_DEBUG_SUBSYS={os.environ.get('NCCL_DEBUG_SUBSYS')}", flush=True)
    print(f"NCCL_IB_DISABLE={os.environ.get('NCCL_IB_DISABLE')}", flush=True)
    print(f"NCCL_IB_HCA={os.environ.get('NCCL_IB_HCA')}", flush=True)
    print(f"NCCL_SOCKET_IFNAME={os.environ.get('NCCL_SOCKET_IFNAME')}", flush=True)
    run("hostname -I || true")
    run("nvidia-smi -L || true")
    run("ls -l /dev/infiniband || true")
    run("for p in /sys/class/infiniband/*/ports/*/state; do echo \"$p=$(cat $p)\";
done 2>/dev/null || true")

    dist.init_process_group(backend="nccl", init_method="env://")

    expected = world_size * (world_size + 1) / 2
    x = torch.full((1024 * 1024,), rank + 1, device=f"cuda:{local_rank}",
dtype=torch.float32)
    for i in range(5):
        y = x.clone()
        dist.all_reduce(y, op=dist.ReduceOp.SUM)
        torch.cuda.synchronize()
        value = float(y[0].item())
        if abs(value - expected) > 1e-5:
            print(f"all_reduce[{i}] validation failed value={value} expected=
{expected}", file=sys.stderr, flush=True)
            sys.exit(2)
        if rank == 0:
            print(f"all_reduce[{i}]= {value} expected={expected}", flush=True)

    dist.barrier()
    dist.destroy_process_group()
    if rank == 0:
        print("PYTORCH_DDP_NCCL_OK", flush=True)
---
apiVersion: batch.volcano.sh/v1alpha1
kind: Job
metadata:
  name: pytorch-2x8-rdma-hca1-demo

```

```
namespace: vcjob-rdma-demo
spec:
  schedulerName: volcano
  queue: default
  minAvailable: 2
  plugins:
    svc: []
    env: []
    pytorch:
      - --master=master
      - --worker=worker
      - --port=23456
  tasks:
    - name: master
      replicas: 1
      policies:
        - event: TaskCompleted
          action: CompleteJob
      template:
        metadata:
          labels:
            app: pytorch-2x8-rdma-hca1-demo
        spec:
          restartPolicy: Never
          schedulerName: volcano
          affinity:
            nodeAffinity:
              requiredDuringSchedulingIgnoredDuringExecution:
                nodeSelectorTerms:
                  - matchExpressions:
                      - key: kubernetes.io/hostname
                        operator: In
                        values:
                          - 10.184.0.44
                          - 10.184.0.46
            podAntiAffinity:
              requiredDuringSchedulingIgnoredDuringExecution:
                - labelSelector:
                    matchLabels:
                      app: pytorch-2x8-rdma-hca1-demo
```

```
    topologyKey: kubernetes.io/hostname
containers:
  - name: trainer
    image: nvcr.io/nvidia/pytorch:24.01-py3
    imagePullPolicy: IfNotPresent
    command: ["bash", "-lc"]
    args:
      - >
        python -m torch.distributed.run
        --nnodes=2
        --nproc_per_node=8
        --node_rank=${RANK}
        --master_addr=${MASTER_ADDR}
        --master_port=${MASTER_PORT}
        /workspace/train.py
    env:
      - name: MASTER_PORT
        value: "23456"
      - name: NCCL_DEBUG
        value: INFO
      - name: NCCL_DEBUG_SUBSYS
        value: INIT,NET
      - name: NCCL_IB_DISABLE
        value: "0"
      - name: NCCL_IB_HCA
        value: mlx5
      - name: NCCL_SOCKET_IFNAME
        value: eth0
      - name: NCCL_NET_GDR_LEVEL
        value: "2"
    resources:
      limits:
        cpu: "32"
        memory: 128Gi
        nvidia.com/gpu: "8"
        rdma/hca: "1"
      requests:
        cpu: "32"
        memory: 128Gi
        nvidia.com/gpu: "8"
```

```
        rdma/hca: "1"
    volumeMounts:
      - name: demo-script
        mountPath: /workspace/train.py
        subPath: train.py
    volumes:
      - name: demo-script
        configMap:
          name: pytorch-2x8-rdma-hca1-demo
- name: worker
  replicas: 1
  template:
    metadata:
      labels:
        app: pytorch-2x8-rdma-hca1-demo
    spec:
      restartPolicy: Never
      schedulerName: volcano
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
              - matchExpressions:
                  - key: kubernetes.io/hostname
                    operator: In
                    values:
                      - 10.184.0.44
                      - 10.184.0.46
      podAntiAffinity:
        requiredDuringSchedulingIgnoredDuringExecution:
          - labelSelector:
              matchLabels:
                app: pytorch-2x8-rdma-hca1-demo
            topologyKey: kubernetes.io/hostname
    containers:
      - name: trainer
        image: nvcr.io/nvidia/pytorch:24.01-py3
        imagePullPolicy: IfNotPresent
        command: ["bash", "-lc"]
        args:
```

```
- >
python -m torch.distributed.run
--nnodes=2
--nproc_per_node=8
--node_rank=${RANK}
--master_addr=${MASTER_ADDR}
--master_port=${MASTER_PORT}
/workspace/train.py
env:
- name: MASTER_PORT
  value: "23456"
- name: NCCL_DEBUG
  value: INFO
- name: NCCL_DEBUG_SUBSYS
  value: INIT,NET
- name: NCCL_IB_DISABLE
  value: "0"
- name: NCCL_IB_HCA
  value: mlx5
- name: NCCL_SOCKET_IFNAME
  value: eth0
- name: NCCL_NET_GDR_LEVEL
  value: "2"
resources:
  limits:
    cpu: "32"
    memory: 128Gi
    nvidia.com/gpu: "8"
    rdma/hca: "1"
  requests:
    cpu: "32"
    memory: 128Gi
    nvidia.com/gpu: "8"
    rdma/hca: "1"
volumeMounts:
- name: demo-script
  mountPath: /workspace/train.py
  subPath: train.py
volumes:
- name: demo-script
```

```
configMap:  
  name: pytorch-2x8-rdma-hca1-demo
```

5. 启动任务

保存 YAML 后执行：

```
kubectl apply -f vcjob-pytorch-2x8-rdma-hca1-demo.yaml
```

查看 Volcano Job：

```
kubectl get vcjob -n vcjob-rdma-demo  
kubectl describe vcjob -n vcjob-rdma-demo pytorch-2x8-rdma-hca1-demo
```

查看 Pod：

```
kubectl get pod -n vcjob-rdma-demo -l volcano.sh/job-name=pytorch-2x8-rdma-hca1-demo -o  
wide
```

正常情况下可以看到 master 和 worker 分别调度到两个节点：

```
pytorch-2x8-rdma-hca1-demo-master-0    Running    10.184.0.46  
pytorch-2x8-rdma-hca1-demo-worker-0    Running    10.184.0.44
```

6. 查看训练日志

查看 master 日志：

```
kubectl logs -n vcjob-rdma-demo pytorch-2x8-rdma-hca1-demo-master-0
```

查看 worker 日志：

```
kubectl logs -n vcjob-rdma-demo pytorch-2x8-rdma-hca1-demo-worker-0
```

提取 NCCL/RDMA 关键日志：

```
kubectl logs -n vcjob-rdma-demo pytorch-2x8-rdma-hca1-demo-master-0 \
  | rg 'NET/IB|Using network|GPU Direct RDMA|all_reduce|PYTORCH_DDP_NCCL_OK|nranks 16'
```

成功时应看到类似日志：

```
NCCL INFO NET/IB : Using [0]mlx5_0:1/IB/SHARP ...
NCCL INFO Using network IBext
NCCL INFO GPU Direct RDMA Enabled
rank 0 nranks 16 cudaDev 0 ... Init COMPLETE
all_reduce[0]=136.0 expected=136.0
all_reduce[1]=136.0 expected=136.0
all_reduce[2]=136.0 expected=136.0
all_reduce[3]=136.0 expected=136.0
all_reduce[4]=136.0 expected=136.0
PYTORCH_DDP_NCCL_OK
```

其中：

- `NET/IB` 表示 NCCL 使用 IB/RDMA 网络。
- `Using network IBext` 表示 NCCL 使用 IB 网络插件。
- `GPU Direct RDMA Enabled` 表示启用了 GPU Direct RDMA。
- `nranks 16` 表示 2 个节点 x 每节点 8 个进程均加入 NCCL 通信域。
- `all_reduce=136.0 expected=136.0` 表示跨 16 个 rank 的通信结果正确。

7. 确认任务完成

查看 Job 状态：

```
kubectl get vcjob -n vcjob-rdma-demo pytorch-2x8-rdma-hca1-demo -o yaml
```

成功时状态中应包含：

```
phase: Completed
```

查看 Pod 状态：

```
kubectl get pod -n vcjob-rdma-demo -l volcano.sh/job-name=pytorch-2x8-rdma-hca1-demo -o wide
```

成功时可看到 Pod 为 `Completed`。如果配置了：

```
policies:
- event: TaskCompleted
  action: CompleteJob
```

master 完成后 Volcano 会将整个 Job 标记完成。不同 Volcano 版本或清理策略下，worker Pod 可能会保留，也可能被清理；应以 VCJob 状态、事件和日志作为最终判断依据。

8. 常见调整项

8.1 修改节点数量

如果要使用更多节点，需要调整：

```
--nnodes=<节点数>
minAvailable: <节点数>
```

并增加对应 task 或 replicas 配置。

8.2 修改每节点 GPU 数

如果每节点使用 4 张 GPU，需要调整：

```
--nproc_per_node=4
nvidia.com/gpu: "4"
```

总 world size 会变为：

```
nnodes x nproc_per_node
```

8.3 修改 RDMA 申请数量

当前示例使用：

```
rdma/hca: "1"
```

该配置已验证可以完成 2 节点 x 8 卡 PyTorch NCCL 分布式训练。

如果需要测试多 HCA 带宽或多轨通信，可以增加该值，例如：

```
rdma/hca: "8"
```

但功能性验证和普通训练启动不一定需要每张 GPU 对应一个 `rdma/hca`。

8.4 限定计算网 IB

如果需要避免使用存储网 IB，需要把 `NCCL_IB_HCA` 改为计算网 HCA 白名单：

```
env:  
  - name: NCCL_IB_HCA  
    value: mlx5_0,mlx5_1,mlx5_2,mlx5_3
```

提交任务后确认日志中只出现这些 HCA：

```
NCCL INFO NET/IB : Using [0]mlx5_0 ... [3]mlx5_3
```

不应出现存储网对应的 `mlx5_*`。

9. 清理任务

如果不再需要保留验证任务，可以删除 namespace：

```
kubectl delete namespace vcjob-rdma-demo
```

如果 namespace 中还有其他资源，只删除当前 VCJob：

```
kubectl delete vcjob -n vcjob-rdma-demo pytorch-2x8-rdma-hca1-demo  
kubectl delete configmap -n vcjob-rdma-demo pytorch-2x8-rdma-hca1-demo
```